

**Case Study: Mobile app  
development of LatinoLink.  
Spanish service  
marketplace for Toronto.**

**Software Engineer:** Carlos López

**Live app:** [App Store](#) · [Google Play](#)

**Timeline:** February – August 2026

**Stack:** FlutterFlow, Flutter/Dart,  
Firebase, Cloud Functions,  
RevenueCat

**Contact:**

[carloslopezr29@gmail.com](mailto:carloslopezr29@gmail.com)

[linkedin.com/in/carloslopezdev](https://linkedin.com/in/carloslopezdev)

**Client**

LatinoLink, a Toronto-based business that commissioned a marketplace connecting the city's Latin American community with trusted service providers.

**Problem**

A marketplace built on trust is only as credible as its enforcement. With the app writing to the database directly, every rule the business depends on, from credit balances to payments to verification, had to be secured where users can't reach it.

**Result**

End-to-end delivery, from requirements to production in seven months

Full system architecture and data model

Server-enforced security with an 81-test adversarial suite

Distance-based lead-matching engine with self-healing recovery

Idempotent in-app payments through RevenueCat

Launch on the App Store and Google Play, now serving 100+ users

# **Index**

- 1. Introduction
- 2. System overview
  - 2.1 User roles and credit model
  - 2.2 Architecture
  - 2.3 Where rules are enforced
- 3. Development process
- 4. Security audit
  - 4.1 Audit criteria
  - 4.2 Balance and status fields
  - 4.3 Lead acceptance
  - 4.4 Membership status
  - 4.5 Phone verification
  - 4.6 Audit findings
- 5. Lead Assignment Engine
  - 5.1 Waterfall vs. broadcast
  - 5.2 Rejection handling and recovery
- 6. App Store requirements
  - 6.1 How App Review shaped the design
  - 6.2 Idempotent payment processing
- 7. Lessons and conclusion
  - 7.1 What I would do differently
  - 7.2 Conclusion
- References

## 1. Introduction

LatinoLink is a Spanish-first service marketplace for the Greater Toronto Area. I designed and built it as the sole engineer for a client, taking it from the first discovery questionnaires all the way to production.

The idea started from a simple observation: Latin Americans in the GTA already find plumbers and electricians through Facebook groups and WhatsApp chains, but those networks can't verify a provider or vouch for the quality of their work. A language barrier makes both worse. Explaining a technical job in a second language is hard, and language is the most frequently reported basis of discrimination among Latin Americans in Canada, ahead of ethnicity and race [1]. A marketplace that replaces these networks is really selling trust, so LatinoLink verifies users by SMS and asks providers for proof of who they are: Ontario licenses, insurance, certifications, and business registration.

My purpose here is to show how that trust is enforced rather than promised. LatinoLink runs on Firebase, a serverless backend chosen to ship fast: the app reads and writes the database directly, without an application server of its own. That choice moves enforcement into two layers: security rules that guard every request and Cloud Functions that handle anything privileged. Every rule the business depends on either executes in one of those layers or can be bypassed by anyone with the app and a little curiosity. That includes whether a provider can change their own credit balance, who may accept a lead, whether a payment is granted exactly once, and who appears in the directory. Deciding where each rule lives was the central engineering problem of the project, and it is the lens this study applies throughout.

The study opens with the system itself: user roles, architecture, and the three places a rule can execute, followed by the development process. At its core is a security audit I ran against my own rules after launch, where three controls held and the fourth gained one more server-side check. From there it covers the lead assignment engine and the changes App Review brought to payments and verification, and it closes with what I would build differently.

---

## 2. System overview

### 2.1 User roles and credit model

LatinoLink has three user types.

**Clients** can browse a directory of Latin businesses and a list of provider services. They verify their phone number by SMS code when they submit their first service request, and providers verify at their first credit purchase, the two points where an unverified number starts costing the marketplace money. To limit spam, each client can hold at most three active requests at a time.

**Providers** can offer multiple services. During onboarding, each provider is asked to upload certification, whether Ontario certification for regulated trades or a certification from their country of origin for non-regulated ones. Providers purchase credit packages through in-app purchase, and those credits are then spent to accept the leads they are offered within range of their location.

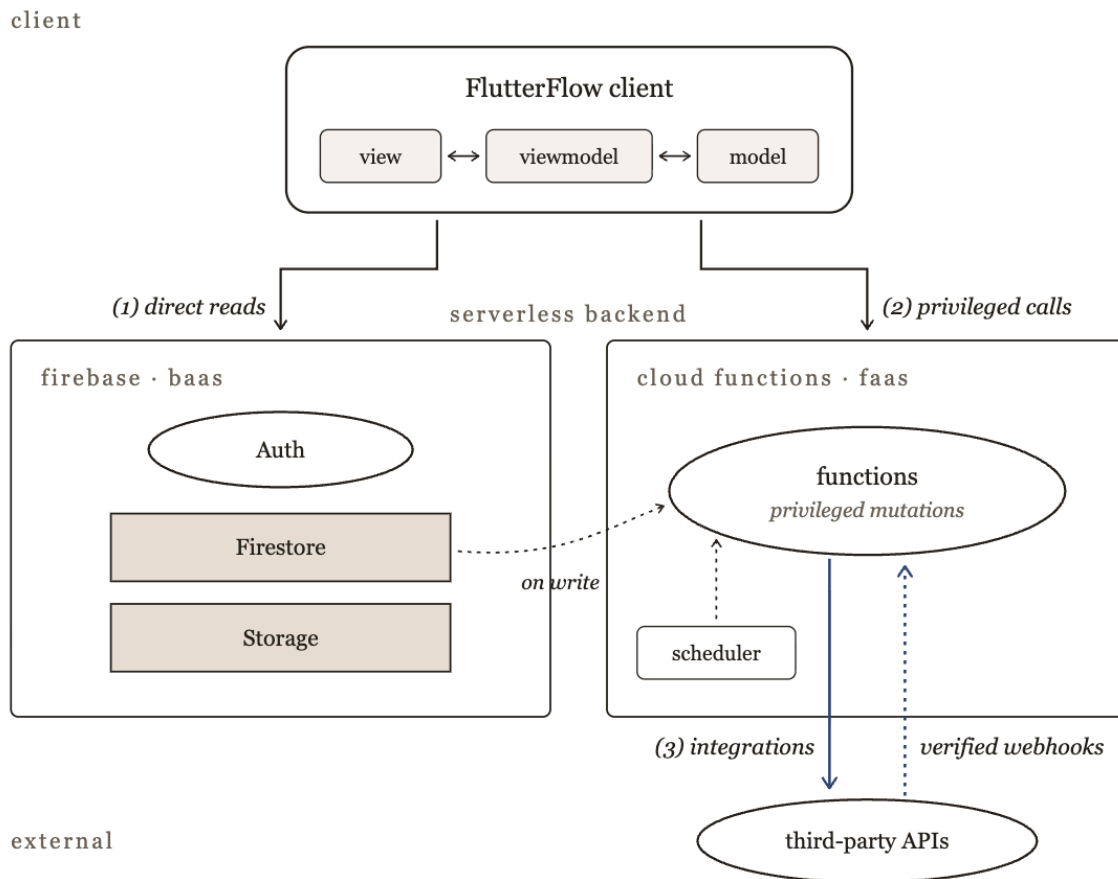
**Business owners** upload their business information and submit a listing, which an administrator reviews before it goes live. The listing category is checked against a membership taxonomy; therefore, a trade profession cannot be used to create a directory listing. Once approved, the owner purchases a monthly or annual membership in order to be featured on the directory shown to clients.

**Credit model.** A client's service request becomes a lead, which moves through the system in one pass. The server geocodes the postal code and sorts eligible providers by Haversine distance, and the closest one gets a single offer with a thirty-minute window. Accepting debits a credit and closes the lead; rejecting or timing out passes it to the next candidate. Only one offer is ever outstanding, and a scheduled sweeper picks up any lead left without one.

Credits are the revenue path on the provider side: they are bought in packages of three through in-app purchase, held as a balance on the user record, and spent at one credit per accepted lead. Memberships are the revenue path on the business side.

## 2.2 Architecture

The client is a FlutterFlow app built from components on an MVVM pattern. The backend is serverless, in two layers: Firebase (Firestore, Auth, and Storage) as the backend-as-a-service layer the app reads directly, and Cloud Functions for privileged writes and third-party integrations. The backend is event-driven, running on Firestore triggers, a scheduler, and verified webhooks.



**Figure 2:** System architecture

## 2.3 Where rules are enforced

Every rule this system asserts executes in one of three locations, and the properties of each location differ:

1. **Security rules.** Firestore evaluates rules on every request, and the rules layer is not reachable from the client. By default it operates at the level of documents and collections. However, custom field immutability operators were written on top of that default in order to reach field-level granularity, which is what keeps sensitive values such as credit balance, membership state, and verification flags from being modified by their own owner.
2. **Cloud Functions.** Functions run on infrastructure the client does not control and carry the privileged logic: lead assignment, in-app purchase handling, and account creation and deletion. They respond to database triggers, callable requests, and verified webhooks. A function only enforces a rule if the client cannot skip it, though, and a security rule denying the direct write is what makes it unskippable. The two layers depend on each other.
3. **The app.** The FlutterFlow client queries the state and gates the interface accordingly, and custom actions written in Dart invoke the privileged Cloud Functions where the visual builder could not reach.

A rule may also be asserted by the product without executing anywhere in the system, in which case it depends entirely on users choosing to comply.

This distinction supplies the criterion that the rest of the study applies. A control holds if it executes in the first two locations, and it does not hold if it executes only in the third or not at all.

---

### 3. Development process

The project ran in three phases, each feeding the next: discovery turned the founder's idea into requirements, iteration turned requirements into a working product, and verification checked that the product enforced what it promised.

**Discovery.** The founder's idea became a requirements list through structured questionnaires and flowcharts, completed in three passes. The loop earned its keep early: the certification requirement reversed between the second and third pass, before any of it was built.

**Iteration.** Development followed a rapid application development model with an agile overlay, where each prototype was built to be revised rather than defended. Figma AI and Claude Design generated the mockups, but none shipped unreviewed, since neither tool produces the same correct result twice. Every deliverable went straight to the founder, and her feedback set the scope of the next iteration. The tools accelerated production; the direct line to the founder kept it pointed at the right requirements.

**Verification.** Testing targeted the authorization boundary, not just the happy path: the emulator suite asserts that unauthorized access and tampering are denied, and payment flows were exercised end to end on TestFlight and Play's internal track, from purchase through deletion, re-registration, and restore. After launch, the same standard was turned on the system itself, which is the review Section 5 reports.

---

### 4. Security audit

After launch, with over 100 users in production, I went back and checked whether the controls actually held. I picked the four that protect the marketplace's money and integrity: credit balances, lead acceptance, membership status, and SMS verification. Each one matching the standard established in Section 2.3: it counts only if it runs somewhere the client cannot reach. Two records made that check possible: the rationale written inline in the rules and functions during development, rejected alternatives included, and an emulator test suite built to attack the authorization boundary, with 63 of its 81 tests asserting that unauthorized access or tampering is

denied. Three controls were held. One did not, and the rest of this section covers how that gap surfaced and how it was closed.

#### 4.1 Audit criteria

Each control is held to two criteria. The first is placement, already set in Section 2.3: a control must run in the security rules, or in a Cloud Function the rules force the client through. Anything left to the client ends up as manual work, and that work grows with every user onboarded.

The second is selection. Locking every field is as much a failure as locking none, since ordinary edits become impossible; fields that carry consequence get locked, and the rest stay open. A control holds only if it meets both.

Each control below backs a promise LatinoLink makes: no user can alter their own balance, role, or status; only the provider a lead was offered to can accept it; only paying businesses appear in the directory; and no unverified number can request a service or buy credits. Each is evaluated against both criteria.

#### 4.2 Balance and status fields

**Promise.** No user can alter their own economic or verification status: a provider cannot touch their credit balance or certificate status, a business owner cannot change their membership tier or featured state, and a client cannot edit a request once created.

**Enforcement.** Firestore rules operate on whole documents and have no notion of the fields inside them, so field-level protection had to be built on top. A custom null-safe helper compares each locked field before and after a write and rejects any change: six fields on user profiles, certificate status on provider profiles, and ten subscription fields on business listings. Three of the user fields are also blocked at creation and initialized instead by a server trigger, so the lock applies from the very first write. Leads skip field-level treatment entirely, since all client updates are denied outright. The emulator suite covers these paths with 63 denial assertions. Fifteen profile fields, display name and notification preferences among them, stay deliberately open.

**Result.** Holds on both criteria: enforced server-side and locked only where a change carries a consequence.

### 4.3 Lead acceptance

**Promise.** A lead is offered to one provider at a time, and only that provider can accept it.

**Enforcement.** Leads are immutable at the rules layer, so acceptance cannot be written directly and must pass through a callable function. Before writing anything, the function checks four conditions: the caller is the lead's current provider, the lead is still assigned, the offer is pending and unexpired, and the caller holds at least one credit. It then updates the lead, the user, and the assignment in a single transaction, so all three succeed or fail together. Balance is checked twice, for different reasons: once during matching, as a fast filter that skips providers who plainly cannot pay, and again at debit time, transactionally, so concurrent offers can never drain a balance below zero.

This is where the two server layers depend on each other. Rules cannot express a transactional read across three documents, and a function guarantees nothing if the client can bypass it; the rules deny the direct write, and that denial makes the function's checks unavoidable. The rules also carry their own share: they block fabricated server state at lead creation, and they limit a lead's visibility to its client, its current provider, and providers previously offered it.

**Result.** Holds: each layer enforces what the other cannot, and neither can be bypassed from the client.

### 4.4 Membership status

**Promise.** Only paying members are featured in the directory.

**Enforcement.** Membership lives in three places: the store receipt, the entitlement in RevenueCat and a cached flag in Firestore that the directory reads for speed. The client cannot write that flag: the membership fields are locked, and the RevenueCat webhook is their only writer. One field stays deliberately open, since the client sets it just before a purchase, and the rule records why.

Unlike the other controls, this one also governs reads. With the same fact held in three places, the question is not only who can change it but also which copy to trust. The cache is never authoritative: when it reads negative, the app checks the entitlement directly and fails closed if that lookup errors. A paying member may briefly see a locked feature, but a non-paying one never sees an unlocked one.

**Result.** Holds, accepting a brief window of stale negatives in exchange for no false positives.

#### 4.5 Phone verification

**Promise.** A client is verified by SMS before requesting a service, and a provider before buying credits.

**Enforcement.** App review requires this kind of data to be collected only when a feature needs it, so verification happens at first request and first purchase rather than at signup. The flag itself is unforgeable: locked against client writes and set only by the verification function, which checks the submitted code against a stored hash under an attempt cap, an expiry, and rate limits. The number is saved only on success.

Running security checks against the rules after launch, I found the gate enforced only in the app's interface; nothing on the server read the flag. The value was secure, but its use was not; a value nothing consumes is inert. The fix went into the lead creation rule: it already reads the caller's user document to confirm their role, so the verification check reuses that same read at no extra cost. Two new emulator tests cover it, bringing the suite to 81 tests and 63 denial assertions.

**Result.** Enforced server-side. Locking a field and gating on it are separate decisions, and every locked field now has a server-side reader.

#### 4.6 Audit findings

Every control protecting money or identity held: credit balances, lead acceptance, and membership state. Verification was the outlier, and the reason is instructive. The criterion for what to lock had been economic consequence, and the verification flag carries consequence without carrying money: it governs eligibility, whether a lead comes from a verified client. It was locked as thoroughly as a balance, but locking a field and gating on it are separate decisions, and only the first had been made on the server.

That distinction is now a design invariant: every locked field has a server-side reader, and every field a server-side check reads is locked. Because it applies to fields that don't exist yet, it keeps locking and gating aligned as the system grows.

None of these rules are platform defaults. Firestore denies by default but has no notion of fields, and FlutterFlow's rule builder could not express six of the twelve collections, so every rule was written by hand and every field chosen deliberately. Credit balance and display name sit on the same document, under the same rule; only one is locked. The result is a system whose security follows its economics: every path that moves money or grants access to it is enforced on the server and backed by an adversarial test suite.

---

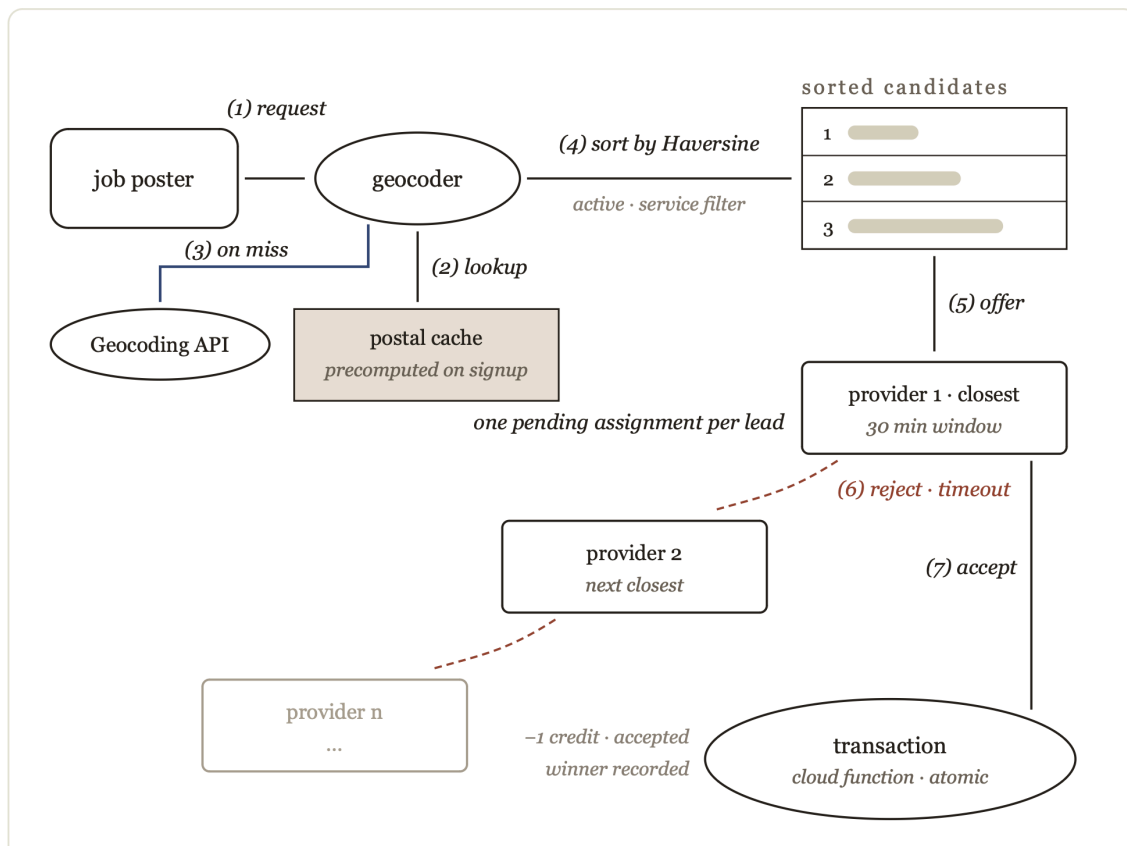
## 5. Lead Assignment Engine

The lead assignment engine carries the app's first revenue stream. When a client requests a service, a Cloud Function runs the matcher, which ranks qualified providers by Haversine distance. Geohash queries on Firestore were the alternative, but at the current scale, computing distance for every qualified provider keeps the matcher simple and easy to reason about; Redis geospatial indexes are the path forward once the provider base outgrows it.

1. The client's postal code is geocoded against a Firestore cache, falling back to the Google Geocoding API on a miss. Provider coordinates are precomputed at signup, so no provider geocoding happens on the request path.
2. Two queries find qualified providers: a collection-group query for those offering the requested service, then a lookup of their user records. The closest provider receives a single pending offer with a thirty-minute window.
3. On acceptance, one transaction debits the credit, marks the offer accepted, and records the provider on the lead. On rejection or timeout, the matcher moves to the next candidate.

### 5.1 Waterfall vs. broadcast

Two models were evaluated: broadcast each lead to every nearby provider and let the first claim win, or offer it to one provider at a time in distance order. The founder's requirement settled it: each provider gets thirty minutes to accept or reject before the lead moves on, and a per-provider window only works if one provider holds the lead at a time. So I chose the waterfall. It is also the more robust model. Broadcast multiplies database load on every lead and creates a race at acceptance, leaving every losing provider with a stale offer to reconcile. The waterfall keeps exactly one pending offer per lead, so accepting or rejecting is a single-record update in one transaction, and a rejection only means finding the next candidate. It also gives providers what they expect: the closest match is always offered first.

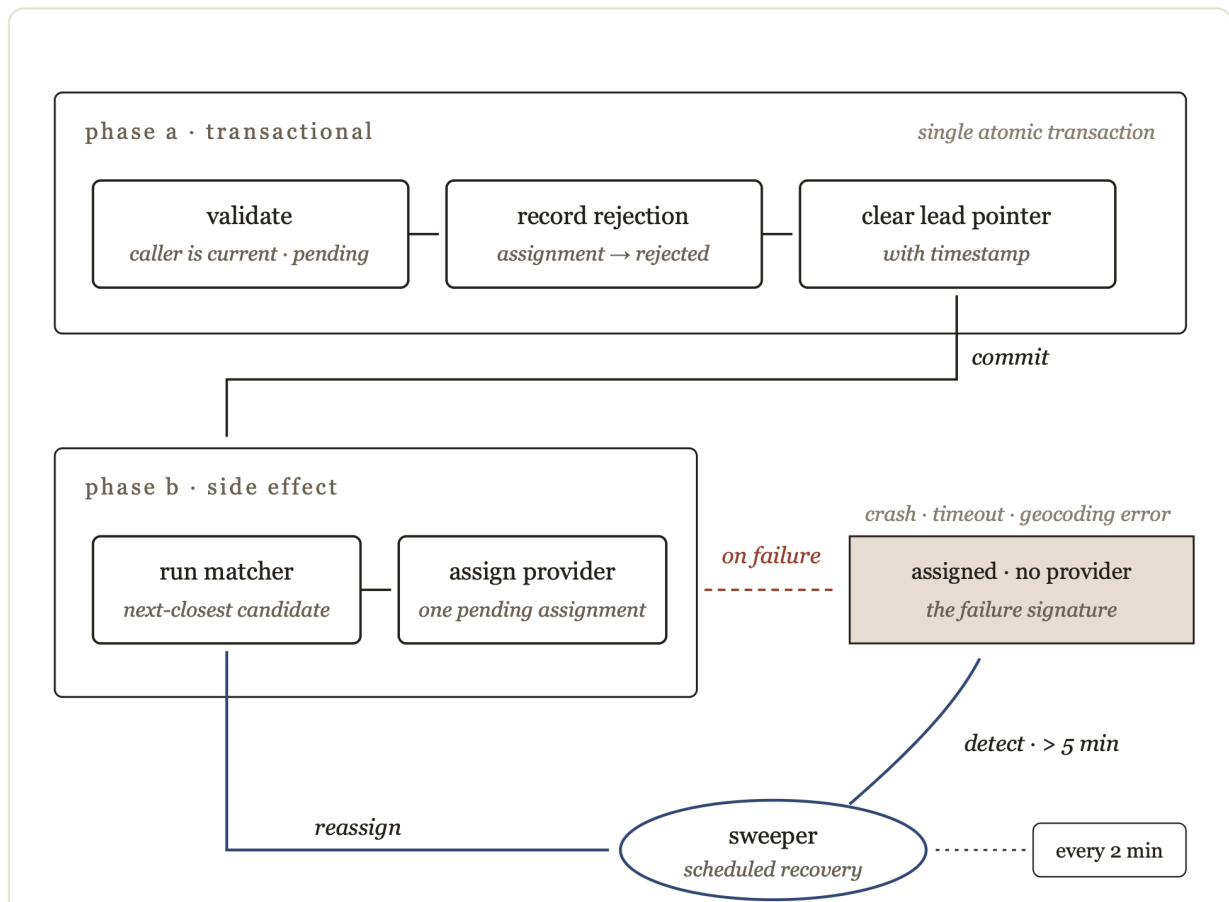


**Figure 3:** Lifecycle of a lead through the waterfall model.

## 5.2 Rejection handling and recovery

Rejecting a lead triggers a search for the next provider, and those queries don't belong inside a transaction, so rejection runs in two phases. Phase A is transactional: it confirms the caller is the current provider and the offer is still pending, records the rejection, and clears the lead's provider pointer with a timestamp. Phase B then runs the matcher outside the transaction.

Everything after Phase A is recoverable by design. An interrupted Phase B leaves exactly one detectable state: a lead marked assigned, with no current provider, for more than five minutes. A sweeper running every two minutes finds that state and reruns the matcher. The timestamp exists for that purpose alone, and it guarantees no lead is ever stranded without an offer.



**Figure 4.** Sweeper and Two-Phase Rejection.

---

## 6. App Store requirements

Design decisions came from three sources: the founder's requirements, my own engineering choices, and App Review. Two of Apple's guidelines reshaped the payment system and the user flow.

### 6.1 How App Review shaped the design

**Payments, Guideline 3.1.1.** Digital goods consumed in an app must be sold through in-app purchase. Credits pay for services delivered outside the app, which argued for an exemption, but the founder's move to add memberships, a purely in-app feature, settled it. Stripe came out of the build, and payments moved to in-app purchase through RevenueCat for cross-platform support, which meant redesigning the webhooks and the APIs around them.

**Verification timing, Guideline 5.1.1.** Personal data can only be collected when a feature requires it, so SMS verification moved from signup to a client's first request and a provider's first credit purchase. Users can now explore before handing over a phone number, and the data model is cleaner: numbers are written only on successful verification, so the users collection holds only verified numbers, and that is what makes the uniqueness check meaningful. A signup gate would have stored a number for every abandoned registration. The constraint raised a question the threat model had not: not whether a check can be bypassed, but why the data is held at all.

### 6.2 Idempotent payment processing

Any endpoint that moves money must be idempotent. RevenueCat's webhook delivers each event at least once, retrying up to five times when no response arrives, so a retry after a lost response would grant a second membership period for a single payment. The handler keys idempotency on RevenueCat's event identifier, claiming it with a create-only write of a document under that ID: the first delivery succeeds, and any repeat fails to create the document and exits without acting.

The store's transaction identifier is the intuitive key, but it silently breaks. On Google Play, a renewal appends an incremented suffix to the base order ID, while an expiration creates no new order and reuses the last one. A guard keyed on it would treat the expiration as already

processed, skip it, and leave the membership active indefinitely. That behavior was confirmed against RevenueCat's transaction records, and the full lifecycle was exercised end to end on TestFlight and Play's internal track: purchase, deletion, re-registration, and restore.

---

## 7. Lessons and conclusion

### 7.1 What I would do differently

**Review store guidelines at the design stage.** Ten review cycles across two versions produced the costliest changes in the project, all architectural: payments moved to in-app purchase, verification moved off signup, and account deletion was built from scratch. Every one was discoverable in Apple's published guidelines before a line of code was written. Checking the full flow against both stores' guidelines up front costs a day and saves weeks.

**Automate the parts of review that aren't judgment.** Certification review has to stay human; chasing providers for missing documents does not. A function that detects what a profile lacks and sends the follow-up would remove that recurring work and leave the decision with a person.

### 7.2 Conclusion

LatinoLink launched to meet the need for security in a digital service marketplace, exceeding the client's expectation of an MVP. What shipped goes further: an adversarial test suite, idempotent payments, a proximity-based lead-matching algorithm, and a post-launch security audit. LatinoLink is live on the App Store and Google Play, and it holds to the standard it was built on: trust enforced by the system, not assumed from its users.

## References

- [1] Statistics Canada, "Study: Portrait of the Latin American populations in Canada," *The Daily*, Jun. 29, 2026. [Online]. Available: <https://www150.statcan.gc.ca/n1/daily-quotidien/260629/dq260629a-eng.htm>